

Yeastar P-Series PBX Integration Guide

System: Global ERP - Call Center Module **PBX:** Yeastar P-Series (OpenAPI v1.0) **Date:** March 2026 **Version:** 2.0

Table of Contents

- 1. [Overview & Architecture](#)
- 2. [Prerequisites](#)
- 3. [Yeastar PBX Configuration](#)
- 4. [Application Configuration Reference](#)
- 5. [Authentication & Token Management](#)
- 6. [API Endpoints Reference](#)
- 7. [Webhook Events & Processing](#)
- 8. [Database Schema](#)
- 9. [Step-by-Step Migration Guide](#)
- 10. [Linkus Browser SDK \(Optional\)](#)
- 11. [Troubleshooting](#)
- 12. [Appendix: Quick Reference Card](#)

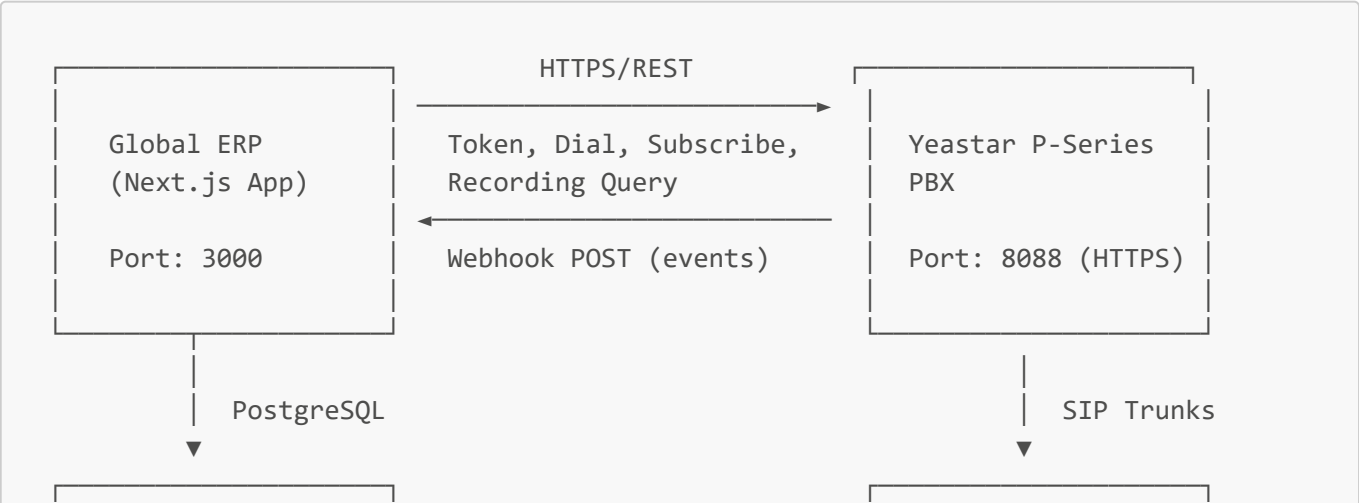
1. Overview & Architecture

System Components

The integration connects a **Yeastar P-Series PBX** to the **Global ERP** application (Next.js / Node.js) to enable:

- **Outbound calling** from the ERP UI (click-to-call)
- **Inbound call popups** with customer identification
- **Call session tracking** with full history
- **Call recording retrieval** linked to call sessions
- **Auto-answer / auto-pickup** for inbound calls
- **Browser-based softphone** via Linkus SDK (optional)

Communication Flow



Database	PSTN / SIP Provider
- integration_dialers	(Outbound/Inbound
- call_sessions	Phone Lines)
- call_recordings	

Authentication Methods Supported

Method	Fields Required	Use Case
Username / Password	username, password	Standard API access via get_token
Client Credentials (OAuth2)	clientId, clientSecret	Preferred for server-to-server
Linkus SDK	accessId, accessKey	Browser softphone integration

2. Prerequisites

Yeastar PBX Requirements

- **Model:** Yeastar P-Series (P550, P560, P570, or Cloud Edition)
- **Firmware:** Version 83.x or higher (OpenAPI v1.0 support)
- **License:** API/Integration license activated
- **OpenAPI:** Enabled in PBX admin panel

Network Requirements

Item	Details
PBX HTTPS Port	8088 (default) - must be accessible from the ERP server
Webhook Connectivity	PBX must be able to reach the ERP webhook URL (HTTP POST)
SSL	Self-signed certificates supported (set sslVerify: false)
Firewall	Allow bidirectional traffic between ERP server and PBX on port 8088

Required Information Before Starting

Gather the following before beginning integration:

1. **PBX IP/Hostname** and HTTPS port (e.g., https://192.168.50.253:8088)
2. **API credentials** (username/password OR client ID/secret)
3. **Default extension number** for outbound calls
4. **ERP server public URL** for webhook callbacks
5. **(Optional)** Linkus SDK Access ID and Access Key

3. Yeastar PBX Configuration

Step 3.1: Enable OpenAPI

1. Log in to the Yeastar PBX web admin panel
2. Navigate to **Settings > General > OpenAPI**
3. Enable **OpenAPI** toggle
4. Note the **API Base URL** displayed (e.g., <https://192.168.50.253:8088/openapi/v1.0>)

Step 3.2: Create API Credentials

Option A: Username/Password

1. Go to **Settings > General > OpenAPI > User Credentials**
2. Create a new API user or use an existing PBX admin account
3. Note the **username** and **password**

Option B: Client Credentials (Recommended)

1. Go to **Settings > General > OpenAPI > Client Credentials**
2. Click **Add**
3. Set a descriptive name (e.g., "Global ERP Integration")
4. Copy the generated **Client ID** and **Client Secret**

Step 3.3: Configure Extensions

1. Go to **Extensions** in the PBX admin
2. Ensure the default extension (e.g., **1000**) exists and is active
3. If using auto-answer, verify the extension supports it

Step 3.4: Configure Outbound Routes (if needed)

1. Go to **Call Control > Outbound Routes**
2. Ensure a route exists that allows the default extension to dial external numbers
3. If using **dialPermission**, note the permission name/value

Step 3.5: Enable Call Recording (Optional)

1. Go to **Call Control > Recording**
2. Enable recording for desired extensions or trunks
3. Recordings will be retrievable via the API

4. Application Configuration Reference

All configuration is stored in the **integration_dialers** table as a JSONB **credentials** column. Below is the complete field reference:

Required Fields

Field	Type	Example	Description
apiBaseUrl	URL	https://192.168.50.253:8088	PBX API base URL with port

Field	Type	Example	Description
defaultExtension	String	1000	Extension used as caller when not specified

Authentication Fields (provide ONE pair)

Field	Type	Description
username	String	API username for get_token endpoint
password	String	API password
clientId	String	OAuth2 Client ID (alternative to username/password)
clientSecret	String	OAuth2 Client Secret

Optional Fields

Field	Type	Default	Description
apiPath	String	openapi/v1.0	API path appended to base URL
autoAnswer	String	yes	Auto-answer on outbound dial (yes / no)
dialPermission	String	(none)	Outbound dial permission name
userAgent	String	OpenAPI	HTTP User-Agent header value
webhookUrl	URL	(none)	Public URL for Yeastar to push events to
sslVerify	Boolean	true	Set false for self-signed PBX certificates

Linkus SDK Fields (Optional - for browser softphone)

Field	Type	Description
linkusServerUrl	URL	Linkus SDK server URL (usually same as PBX URL)
linkusDefaultExtension	String	Auto-filled extension for browser SDK login
accessId	String	Yeastar Linkus SDK Access ID
accessKey	String	Yeastar Linkus SDK Access Key

5. Authentication & Token Management

Token Request Flow



```
sequenceDiagram
    participant Client
    participant Server
    Client->>Server: user_agent: "OpenAPI" }
    Server-->>Client: Response: {
    errcode: 0,
    access_token: "abc123...",
    expires_in: 3600
    }
    Note over Client: (Token cached for ~1 hour)
```

(Token cached for ~1 hour)

Token Request - Username/Password

```
POST https://<PBX_IP>:8088/openapi/v1.0/get_token
Content-Type: application/json
User-Agent: OpenAPI

{
  "username": "api_user",
  "password": "api_password",
  "user_agent": "OpenAPI"
}
```

Token Request - Client Credentials

```
POST https://<PBX_IP>:8088/openapi/v1.0/get_token
Content-Type: application/json
User-Agent: OpenAPI

{
  "client_id": "your_client_id",
  "client_secret": "your_client_secret",
  "user_agent": "OpenAPI"
}
```

Successful Response

```
{
  "errcode": 0,
  "errmsg": "SUCCESS",
  "access_token": "eyJhbGciOi...",
  "expires_in": 3600
}
```

Token Caching Rules

- Tokens are cached in memory with a key based on the integration ID
- Cache expires **60 seconds before** the actual token expiry (safety margin)
- Default expiry: **3600 seconds** (1 hour) if `expires_in` is not provided
- On error code `10004` (token expired), the cache is cleared and a fresh token is requested automatically

6. API Endpoints Reference

All endpoints use the base URL: `https://<PBX_IP>:8088/openapi/v1.0`

All authenticated requests require the header:

```
Authorization: <access_token>
User-Agent: OpenAPI
```

6.1 Get Token

Endpoint	<code>POST /get_token</code>
Auth	None (this obtains the token)
Purpose	Authenticate and receive an access token

Request Body (username/password):

```
{
  "username": "api_user",
  "password": "api_password",
  "user_agent": "OpenAPI"
}
```

Request Body (client credentials):

```
{
  "client_id": "your_client_id",
  "client_secret": "your_client_secret",
  "user_agent": "OpenAPI"
}
```

Response:

```
{
  "errcode": 0,
```

```
"errmsg": "SUCCESS",
"access_token": "token_string",
"expires_in": 3600
}
```

6.2 Place Outbound Call (Dial)

Endpoint	POST /call/dial
Auth	Authorization: <token>
Purpose	Initiate an outbound call from an extension to a phone number

Request Body:

```
{
  "caller": "1000",
  "callee": "0501234567",
  "auto_answer": "yes",
  "outbound_params": {
    "dial_permission": "international"
  }
}
```

Parameter	Required	Description
caller	Yes	PBX extension number initiating the call
callee	Yes	Destination phone number
auto_answer	No	"yes" or "no" - auto-answer on agent phone
outbound_params.dial_permission	No	Named dial permission on the PBX

Success Response:

```
{
  "errcode": 0,
  "errmsg": "SUCCESS",
  "call_id": "1648281234.42"
}
```

6.3 Query Call Status

Endpoint	POST /call/query
----------	------------------

Auth	Authorization: <token>
-------------	------------------------

Purpose	Query the current status of an active call
----------------	--

Request Body:

```
{
  "call_id": "1648281234.42"
}
```

6.4 Accept Inbound Call (Auto-Pickup)

Endpoint	POST /call/accept_inbound
-----------------	---------------------------

Auth	Authorization: <token>
-------------	------------------------

Purpose	Auto-answer a ringing inbound call on an extension
----------------	--

Request Body:

```
{
  "call_id": "1648281234.42"
}
```

6.5 Subscribe to Webhook Events

Endpoint	POST /subscribe
-----------------	-----------------

Auth	Authorization: <token>
-------------	------------------------

Purpose	Register a webhook URL to receive call and CDR events
----------------	---

Request Body:

```
{
  "url": "https://your-erp-server.com/api/webhooks/dialer/yeastar",
  "user_agent": "OpenAPI",
  "events": [
    { "type": "call" },
    { "type": "cdr" }
  ]
}
```


Important Notes:

- The webhook URL must be publicly accessible from the PBX
- Subscription is refreshed during health checks
- Subscribe to both **call** (real-time events) and **cdr** (post-call records)

6.6 Query Recordings

Multiple endpoints are attempted as fallbacks:

Endpoint	Purpose
POST /recording/query	Query recording by call_id
POST /recording/list	List recordings with filters
POST /record/query	Alternative recording query
POST /record/list	Alternative recording list
POST /cdr/query	CDR query (may include recording URL)
POST /cdr/list	CDR list with time-range filtering

Recording Query Example:

```
{
  "call_id": "1648281234.42"
}
```

Recording List Example (time-range):

```
{
  "start_time": "2026-03-28 10:00:00",
  "end_time": "2026-03-28 11:00:00"
}
```

The system uses a **scoring algorithm** to match recordings:

- Call ID exact match: **100 points**
- From number match: **40 points**
- To number match: **20 points**
- Time proximity (±5 min): **35 points**
- Recording with highest score is selected

7. Webhook Events & Processing

Webhook Endpoint

```
POST /api/webhooks/dialer/yeastar
```

The ERP application exposes this endpoint to receive events pushed by the Yeastar PBX.

Event Types

Yeastar sends events with a numeric **type** field. The system processes four event types:

Type 30011 - Active Call Event

When: During a live call - rings, alerts, answers, hangups **Contains:** Real-time member status updates

Payload Structure:

```
{
  "type": 30011,
  "msg": "{ \"call_id\": \"1648281234.42\", \"type\": \"Inbound\", \"members\":
  [{ \"extension\":
  { \"number\": \"1000\", \"member_status\": \"RING\" } } ], \"from_number\": \"0501234567\",
  \"to_number\": \"1000\" }"
}
```

Member Status Values and Mapping:

member_status	ERP Status	Description
RING	ringing	Extension is ringing
ALERT	ringing	Extension is alerting
ANSWER	ANSWERED	Call has been answered
BYE	completed	Call has ended

Type 30012 - CDR Event (Post-Call)

When: After a call ends - contains the final call detail record **Contains:** Complete call summary with duration and disposition

Payload Structure:

```
{
  "type": 30012,
  "msg": "{ \"call_id\": \"1648281234.42\", \"from_number\": \"0501234567\", \"to_number\": \"1000
  \", \"duration\": \"45\", \"status\": \"ANSWERED\", \"start_time\": \"2026-03-28
```

```
10:15:30\", \"end_time\": \"2026-03-28 10:16:15\"}"}
}
```

Always mapped to status: **completed** (regardless of disposition)

Type 30016 - Inbound Call Request

When: When a new inbound call arrives before routing **Contains:** Initial call information for popup display

Payload Structure:

```
{
  "type": 30016,
  "msg": "
  {\"call_id\": \"1648281234.42\", \"from_number\": \"0501234567\", \"status\": \"Incomin
  g call request\"}"
}
```

Mapped to status: **incoming**

Type 30020 - PCIR Event (Per-Call Info Record)

When: Per-extension call lifecycle events **Contains:** Operation-based updates for individual call legs

Payload Structure:

```
{
  "type": 30020,
  "msg": "{ \"call_id\": \"uuid-
  string\", \"operation\": \"call_start\", \"from_number\": \"0501234567\", \"to_number\"
  : \"1000\", \"type\": \"Inbound\"}"
}
```

Operation Mapping:

operation	ERP Status	Description
call_start	initiated	Call leg started
call_answer	ANSWERED	Call leg answered
call_over	completed	Call leg ended

Filtering Logic: Type 30020 events with unknown caller and short extension targets (1-6 digits) are ignored to prevent duplicate ring-group shadow events.

Direction Detection Logic

The system determines call direction using this priority:

1. If `msg.type` contains "outbound" or direction field contains "out" → **outbound**
2. If `msg.type` equals "External" (type 30020) → **outbound** (extension→PSTN)
3. If `msg.type` contains "inbound" or event type is 30011/30016 → **inbound**
4. **Fallback heuristic** for type 30020: if `from` is ≤5 digits and `to` is ≥7 digits → **outbound**

8. Database Schema

Table: `integration_dialers`

Stores the Yeastar connection configuration.

```
CREATE TABLE integration_dialers (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  provider    text NOT NULL,          -- 'yeastar'
  label       text NOT NULL,          -- Display name (e.g., 'Office PBX')
  auth_type   text NOT NULL,          -- 'api_key'
  credentials jsonb NOT NULL DEFAULT '{}', -- All config fields (see Section 4)
  metadata    jsonb NOT NULL DEFAULT '{}', -- Additional metadata
  webhooks    jsonb NOT NULL DEFAULT '{}', -- Webhook configuration
  is_global   boolean NOT NULL DEFAULT FALSE,
  company_id  uuid NULL,              -- NULL if global, company UUID if
  scoped
  is_active   boolean NOT NULL DEFAULT TRUE,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now()
);
```

Example INSERT:

```
INSERT INTO integration_dialers (provider, label, auth_type, credentials,
is_active, company_id)
VALUES (
  'yeastar',
  'Main Office PBX',
  'api_key',
  '{
    "apiBaseUrl": "https://192.168.50.253:8088",
    "apiPath": "openapi/v1.0",
    "defaultExtension": "1000",
    "username": "api_user",
    "password": "secure_password",
    "autoAnswer": "yes",
    "sslVerify": false,
    "webhookUrl": "https://erp.example.com/api/webhooks/dialer/yeastar",
    "userAgent": "OpenAPI"
  }',
  true,
  null
);
```

```
}',
true,
'your-company-uuid-here'
);
```

Table: `call_sessions`

Tracks every call made or received through the system.

```
CREATE TABLE call_sessions (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  scope             text NOT NULL,          -- 'global' | 'company'
  company_id        uuid NULL,
  branch_id         uuid NULL,
  created_by_user_id uuid NOT NULL,
  direction         text NOT NULL,          -- 'outbound' | 'inbound'
  from_number       text NOT NULL,
  to_number         text NOT NULL,
  to_entity_type    text NULL,              -- 'customer' | 'employee' | 'vendor'
  | 'other'
  to_entity_id      uuid NULL,
  provider_key      text NOT NULL,          -- 'yeastar'
  provider_call_id  text NULL,              -- Yeastar call_id
  status           text NOT NULL,          -- 'initiated' | 'ringing' |
  'in_progress' | 'completed' | 'failed' | 'cancelled'
  started_at       timestampz NULL,
  ended_at         timestampz NULL,
  duration_seconds integer NULL,
  metadata         jsonb NOT NULL DEFAULT '{}',
  created_at       timestampz NOT NULL DEFAULT now(),
  updated_at       timestampz NOT NULL DEFAULT now()
);
```

Table: `call_recordings`

Links recordings to call sessions.

```
CREATE TABLE call_recordings (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  call_session_id   uuid NOT NULL REFERENCES call_sessions(id) ON DELETE
  CASCADE,
  provider_recording_id text NOT NULL,
  url              text NOT NULL,
  duration_seconds integer NULL,
  created_at       timestampz NOT NULL DEFAULT now()
);
```

9. Step-by-Step Migration Guide

Follow these steps to replicate the Yeastar integration on a new system.

Step 1: Set Up the Database

Run the migration scripts to create required tables:

```
# Run migrations in order
psql -d your_database -f migrations/008_call_center.sql
psql -d your_database -f migrations/031_channel_and_dialer_integrations.sql
```

Step 2: Configure the Yeastar PBX

1. Enable OpenAPI on the PBX (see Section 3.1)
2. Create API credentials (see Section 3.2)
3. Ensure extensions and outbound routes are configured

Step 3: Create the Integration Record

Insert the dialer configuration into the database:

```
INSERT INTO integration_dialers (provider, label, auth_type, credentials,
is_active, company_id)
VALUES (
  'yeastar',
  'Office PBX',
  'api_key',
  '{
    "apiBaseUrl": "https://YOUR_PBX_IP:8088",
    "apiPath": "openapi/v1.0",
    "defaultExtension": "1000",
    "clientId": "YOUR_CLIENT_ID",
    "clientSecret": "YOUR_CLIENT_SECRET",
    "autoAnswer": "yes",
    "sslVerify": false,
    "webhookUrl": "https://YOUR_ERP_URL/api/webhooks/dialer/yeastar",
    "userAgent": "OpenAPI"
  }',
  true,
  'YOUR_COMPANY_UUID'
);
```

Step 4: Implement the Webhook Endpoint

Create a POST endpoint at </api/webhooks/dialer/yeastar> that:

1. Receives JSON payloads from the PBX
2. Parses the `type` field to determine event type (30011, 30012, 30016, 30020)
3. Extracts the `msg` field (JSON string) and parses it
4. Maps fields to your internal call session format
5. Creates/updates `call_sessions` records

Minimal webhook handler pseudocode:

```

async function handleYeastarWebhook(request) {
  const payload = await request.json();
  const eventType = Number(payload.type);
  const msg = JSON.parse(payload.msg || '{}');

  const callId = msg.call_id || msg.unique_id;
  const fromNumber = msg.from_number || msg.caller;
  const toNumber = msg.to_number || msg.callee;

  switch (eventType) {
    case 30016: // Incoming call request
      await createCallSession({ callId, fromNumber, toNumber, status: 'ringing',
direction: 'inbound' });
      // Trigger popup notification to agent
      break;

    case 30011: // Active call event
      const memberStatus = msg.members?.[0]?.extension?.member_status;
      if (memberStatus === 'ANSWER') {
        await updateCallSession(callId, { status: 'in_progress' });
      } else if (memberStatus === 'BYE') {
        await updateCallSession(callId, { status: 'completed' });
      }
      break;

    case 30012: // CDR (post-call)
      await updateCallSession(callId, {
        status: 'completed',
        duration: msg.duration,
        startedAt: msg.start_time,
        endedAt: msg.end_time
      });
      // Trigger recording resolution (with 30s delay for processing)
      break;

    case 30020: // PCIR
      // Handle per-extension events (call_start, call_answer, call_over)
      // Filter out ring-group shadow events
      break;
  }

  return new Response('OK', { status: 200 });
}

```

Step 5: Implement Token Management

```
const tokenCache = new Map();

async function getToken(integration) {
  const cached = tokenCache.get(integration.id);
  if (cached && cached.expiresAt > Date.now() + 60000) {
    return cached.token;
  }

  const { apiBaseUrl, apiPath, username, password, clientId, clientSecret } =
    integration.credentials;
  const baseUrl = `${apiBaseUrl}/${apiPath || 'openapi/v1.0'}`;

  const body = username && password
    ? { username, password, user_agent: 'OpenAPI' }
    : { client_id: clientId, client_secret: clientSecret, user_agent: 'OpenAPI' };

  const response = await fetch(`${baseUrl}/get_token`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(body)
  });

  const data = await response.json();
  if (data.errcode !== 0) throw new Error(data.errmsg);

  tokenCache.set(integration.id, {
    token: data.access_token,
    expiresAt: Date.now() + (data.expires_in || 3600) * 1000
  });

  return data.access_token;
}
```

Step 6: Implement Outbound Calling

```
async function placeCall(integration, extension, phoneNumber) {
  const token = await getToken(integration);
  const baseUrl =
    `${integration.credentials.apiBaseUrl}/${integration.credentials.apiPath ||
    'openapi/v1.0'}`;

  const response = await fetch(`${baseUrl}/call/dial`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': token,
      'User-Agent': 'OpenAPI'
    },
  },
```



```
    body: JSON.stringify({
      caller: extension || integration.credentials.defaultExtension,
      callee: phoneNumber,
      auto_answer: integration.credentials.autoAnswer || 'yes'
    })
  });

  const data = await response.json();

  // Handle token expiry (errcode 10004) - clear cache and retry
  if (data.errcode === 10004) {
    tokenCache.delete(integration.id);
    return placeCall(integration, extension, phoneNumber); // Retry once
  }

  return data;
}
```

Step 7: Subscribe to Webhook Events

```
async function subscribeWebhooks(integration) {
  const token = await getToken(integration);
  const baseUrl =
    `${integration.credentials.apiUrl}/${integration.credentials.apiPath ||
    'openapi/v1.0'}`;

  await fetch(`${baseUrl}/subscribe`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': token,
      'User-Agent': 'OpenAPI'
    },
    body: JSON.stringify({
      url: integration.credentials.webhookUrl,
      user_agent: 'OpenAPI',
      events: [
        { type: 'call' },
        { type: 'cdr' }
      ]
    })
  });
}
```

Step 8: Test the Integration

Health Check Test:

1. Obtain a token using `get_token` endpoint
2. If successful, the PBX is reachable and credentials are valid

3. Subscribe to webhook events

Outbound Call Test:

1. Call `POST /call/dial` with a test extension and phone number
2. Verify the extension rings (or auto-answers)
3. Verify a `call_sessions` record is created

Inbound Call Test:

1. Call the PBX DID number from an external phone
2. Verify webhook receives type 30016 (incoming) event
3. Verify type 30011 events fire as the call progresses
4. Verify type 30012 CDR event fires after hangup
5. Verify `call_sessions` record shows complete lifecycle

Recording Test:

1. Make a test call with recording enabled
2. Wait 30 seconds after call ends
3. Query `POST /recording/query` with the `call_id`
4. Verify recording URL is returned and accessible

Step 9: Configure Recording Resolution (Optional)

The system uses deferred recording resolution with retries:

1. **Immediate attempt** (0ms delay) after CDR event
2. **Retry** at 30 seconds if recording not found
3. Query multiple endpoints as fallback (`recording/query` → `recording/list` → `cdr/query` → `cdr/list`)
4. Time-window matching: ±5 minutes around call start/end

10. Linkus Browser SDK (Optional)

For browser-based softphone functionality:

Setup

1. Obtain Linkus SDK credentials from Yeastar PBX admin panel
2. Add the Linkus SDK JavaScript bundle to your web application
3. Configure the integration with `linkusServerUrl`, `accessId`, and `accessKey`

Configuration Fields

```
{
  "linkusServerUrl": "https://192.168.50.253:8088",
  "linkusDefaultExtension": "1000",
  "accessId": "your_access_id",
```

```
"accessKey": "your_access_key"
}
```

SDK Sign Endpoint

The ERP provides endpoints for SDK authentication:

```
POST /api/company/{companyId}/dialer/linkus-sign
POST /api/mobile/company/{companyId}/dialer/linkus-sign
GET  /api/company/{companyId}/dialer/linkus-settings
```

Browser SDK Detection

The system looks for the Linkus SDK in the global scope under these names:

- YSWebRTC
- LinkusWebRTC
- LinkusSDK
- Linkus
- linkus

11. Troubleshooting

Common Issues

Issue	Cause	Solution
Token request fails	Wrong credentials or PBX unreachable	Verify IP, port, username/password or client ID/secret
Error code 10004	Token expired	System auto-retries; check if cache is working
SSL certificate error	Self-signed certificate	Set <code>sslVerify: false</code> in credentials
Webhook not receiving events	PBX cannot reach ERP URL	Ensure webhook URL is publicly accessible; check firewall
"Max limitation exceeded"	Too many token requests	System implements 60-second backoff; reduce request frequency
Duplicate call entries	Ring-group shadow events	Type 30020 events with unknown caller + short extension are auto-filtered
Dial returns error	Extension offline or no outbound route	Verify extension is registered and outbound route allows dialing
Recordings not found	Recording not yet processed	System retries at 30s; check if recording is enabled on PBX

Error Codes

Yeastar Error Code	Meaning	Action
0	Success	No action needed
10004	Token expired/invalid	Clear cache, request new token
10006	Parameter error	Check request body format
10007	Resource not found	Verify extension/call_id exists
10009	Permission denied	Check API user permissions
10023	Max limitation exceeded	Wait 60 seconds, retry

Webhook Debug Logging

The system logs all webhook activity to:

```
/tmp/global-erp/webhook-dialer.log
```

To enable verbose logging, check this file for incoming payloads and processing results.

12. Appendix: Quick Reference Card

API Endpoints Summary

Action	Method	Endpoint
Get Token	POST	/openapi/v1.0/get_token
Place Call	POST	/openapi/v1.0/call/dial
Query Call	POST	/openapi/v1.0/call/query
Accept Inbound	POST	/openapi/v1.0/call/accept_inbound
Subscribe Events	POST	/openapi/v1.0/subscribe
Query Recording	POST	/openapi/v1.0/recording/query
List Recordings	POST	/openapi/v1.0/recording/list
Query CDR	POST	/openapi/v1.0/cdr/query
List CDR	POST	/openapi/v1.0/cdr/list

Webhook Event Types

Type	Name	Status Mapping
30011	Active Call	RING→ringing, ANSWER→answered, BYE→completed

Type	Name	Status Mapping
30012	CDR	Always→completed
30016	Incoming Request	→incoming
30020	PCIR	call_start→initiated, call_answer→answered, call_over→completed

Required Headers (Authenticated Requests)

```
Authorization: <access_token>
Content-Type: application/json
User-Agent: OpenAPI
```

Minimum Configuration Checklist

- ☐ PBX IP/hostname and port (8088)
- ☐ API credentials (username/password OR client ID/secret)
- ☐ Default extension number
- ☐ Webhook URL (publicly accessible)
- ☐ SSL verification setting
- ☐ Database tables created (migrations 008, 031)
- ☐ Webhook endpoint implemented and deployed
- ☐ Event subscription registered
- ☐ Outbound call tested
- ☐ Inbound call tested
- ☐ Recording retrieval tested

End of Yeastar P-Series PBX Integration Guide